

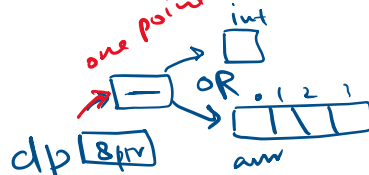
Dynamic 2D Array:

- ① Array of pointers → Heap
- ② Assign address of 1-D Arrays to each pointer.

Multiple in direction.

```
int **dp = new int*;
int **dp = new int*[5];
```

↑ value int variable



Stack
`int *a_ptr[10];`
`int x=10;`
`int *p=&x;`
`int **dp=&p;`
`int **tp=&dp;`

↑ heap

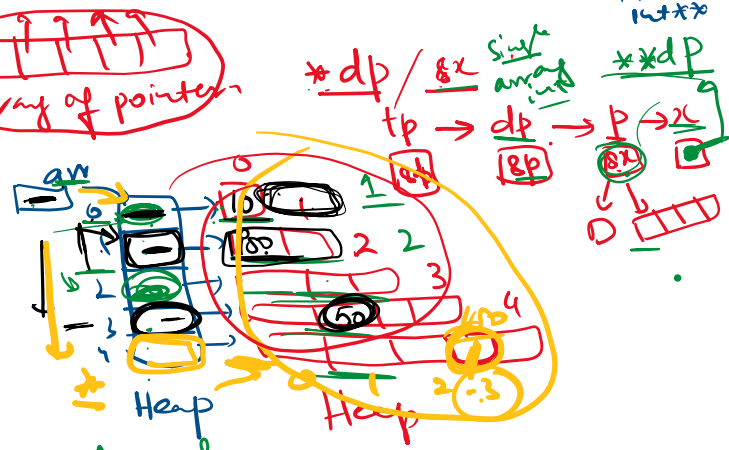
```
int size = 5;
int **arr = new int*[size];
```

```
for(int i=0; i<size; i++)
    arr[i] = new int[1];
```

↑ Array of pointers.

`*arr` → `arr == &arr[0]`
`*(arr+0)` → `arr[0]`
`*(arr+1)` → `arr[1]`
`*(arr+2)` → `arr[2]`

↑ left / right



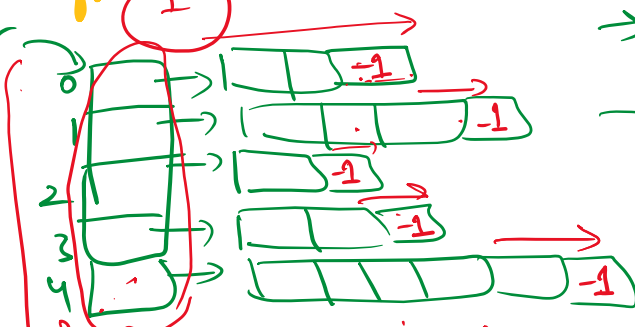
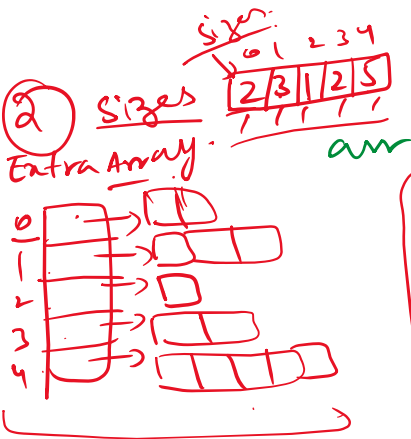
```
**arr = 10;
*(arr+1) = 100;

arr[3][1] = 50;
*(arr+3)+1 = 50;
```

Subscript 1
`[]. 1`
`* deref`
`- +` 3 Add/Sub
 ↑ change precedence
 use `()`
 small.

```
*(arr+4)+3 = 100;
arr[4][3] = 100;
```

Array of point
 Array of integer



```
for(int i=0; i<size; i++)
    for(int j=0; j<arr[i]; j++)
        cout<<arr[i][j];
```

0 — size-1
 → 1 2 3 4 5
 → [2 3 1 2 5]
 Store
`arr[i][j] != -1`

extend!
 → increase Number of rows.
 → decrease → shrink.

void Shrink(arr[4], Size[4]);

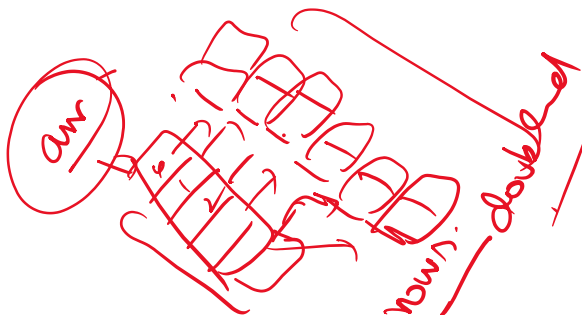
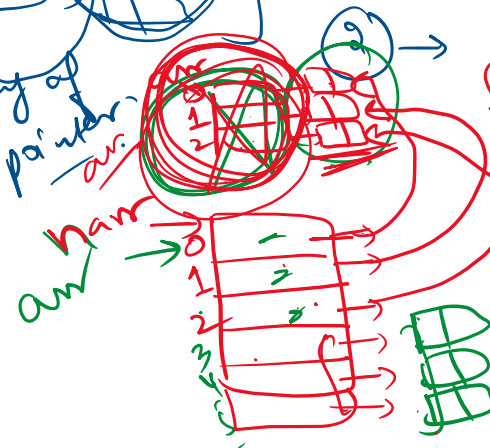
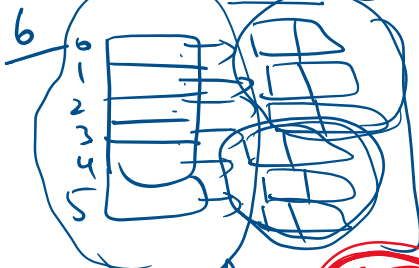
int **arr; Size [2] [3]

→ extend(*arr, Size[0]);
 extend(*arr+1, Size[1]);
 extend(*arr+2, Size[2]);

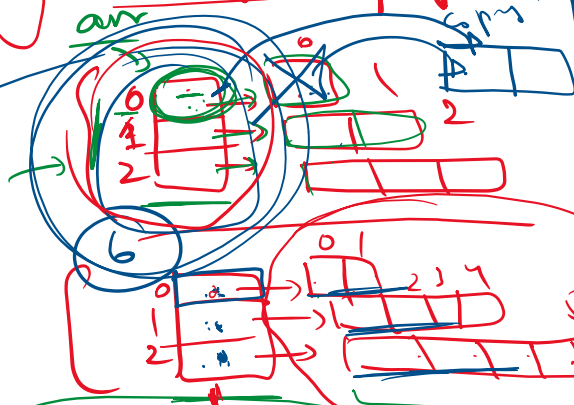
extend(arr[0], Size[0]);
 extend(arr[1], Size[1]);

for(int i=0; i<Size; i++)
 extend(arr[i], Size[i]);

size = 3 rows.
 arr → 2 columns



Arrays of pointers.



Number of columns.

Double content arrays

double

void extend(int **arr, int Size);

new array double size.
 copy content.
 update pointer.
 delete old array.

int Size = 3;
 int *p = new int [Size];
 extend(p, Size);

void extend rows(int **arr, int Size);

int Size = 3;

① int **newarr = new int*[Size*2];

② Copy data from old array (Address)
 for(int i=0; i<Size; i++)
newarr[i] = arr[i];

③ delete [] arr;

arr = newarr;

for(int i=Size; i<Size*2; i++)
 (→ arr[i] = null ptr;
 → arr[i] = new int[2];

update size

④ Size = Size*2;

Deallocation

Reverse order allocation-

1) Arrays of integer data:

①

for(int i=0; i<size; i++)

delete [] arr[i];

② Array of pointer.

②

delete [] arr;

arr = nullptr;

